

Szakdolgozat: Saját programozási nyelv írása

Készítette: Fehér Vivien Fanni

Tartalomjegyzék

Szakdolgozat: Saját programozási nyelv írása.....	3
1. Bevezetés és célkitűzés.....	3
2. A Programozási nyelvek felépítése.....	3
2.1. Lexikális Elemzés - Lexing:.....	3
2.2. Szintaktikai Elemzés - Parsing:.....	4
2.3. Kiértékelés - Evaluation:.....	4
3. A "MiniNyelv" Tervezése és Funkciói.....	4
3.1. Típusrendszer:.....	5
3.2. Változók:.....	5
3.3. Parancsok és Kifejezések:.....	5
3.4. Vezérlési Szerkezetek (IF):.....	5
3.5. Kommentek:.....	6
4. Implementáció Java nyelven:.....	6
4.1. Az Architektúra Fő Komponensei:.....	6
4.2. Tervezési Döntések:.....	8
4.3. A Projekt Osztályszerkezete és Fájljai:.....	9
A. Rendszerkomponensek (Core System):.....	9
B. Interfészek (Interfaces):.....	10
C. Utasítás Implementációk (Statements):.....	10
D. Kifejezés Implementációk (Expressions):.....	11
4.4. Memóriakezelés és Állapottér (Environment):.....	12
4.4.1. A Változók Tárolása.....	12
4.4.2. Élettartam (Scope):.....	14
5. A Rendszer Fordítása és Futtatása.....	14
5.1. Könyvtárszerkezet:.....	14
5.2. Az Interpreter Fordítása (Compilation):.....	15
5.3. A Mini Program Futtatása (Execution):.....	15
5.4. Várt Kimenet.....	16
6. Összegzés és további fejlesztési lehetőségek.....	16

Szakdolgozat: Saját programozási nyelv írása

1. Bevezetés és célkitűzés

A szoftverfejlesztés világában ritkán adódik lehetőség arra, hogy a legalapvetőbb építőkövekkel, magukkal a programozási nyelvekkel foglalkozzunk. A szakdolgozatom egy ilyen folyamatot mutat be, egy saját, minimalista programozási nyelv ("MiniNyelv") tervezését és megvalósítását.

A szakdolgozatom elsődleges célja nem egy ipari erősségű, általános célú programozási nyelv létrehozása volt, hanem **a nyelvkészítés folyamatának és alapelveinek gyakorlati bemutatása**. A célom egy olyan **egyszerű, "script" jellegű nyelv** megírása volt, **amely képes alapvető matematikai és logikai műveletek elvégzésére, változók kezelésére és vezérlési szerkezetek használatára**.*****

A projekt során megvalósított interpreter Java nyelven íródott, kihasználva a Java objektumorientált képességeit, különösen az **Interpreter Tervezési Mintát** (Interpreter Design Pattern).

2. A Programozási nyelvek felépítése

Mielőtt rátérnénk a saját nyelvem implementációjára, fontos megérteni, hogyan kel életre egy programozási nyelv a beírt szöveges kódból. Ez a folyamat szinte mindig három fő lépésből áll:

2.1. Lexikális Elemzés - Lexing:

A "lexer" (vagy "tokenizáló") feladata, hogy a nyers forráskódot (egy hosszú karakterfolyamot) apró, jelentéssel bíró egységekre, úgynevezett **tokenekre** bontsa.

- **Példa:** A `var x = 10;` kódsor a lexer számára nem csak egy szöveg.
- **Eredmény (Tokenek):** `VAR` (kulcsszó), `IDENTIFIER("x")` (azonosító), `EQUALS` (operátor), `NUMBER(10)` (literál), `SEMICOLON` (elválasztó).

2.2. Szintaktikai Elemzés - Parsing:

A "parser" ("elemző") feladata, hogy a lexer által generált tokenek sorozatát fogadja, és ellenőrizze, hogy azok a nyelv nyelvtani szabályainak (a szintaxisnak) megfelelnek-e. Ha igen, a tokenekből egy fa-struktúrát épít fel, amelyet **Absztrakt Szintaxis Fának (AST - Abstract Syntax Tree)** nevezünk.

- **Példa:** A $5 + (2 * 3)$ kifejezés AST-je:

```
      +
     /\
    5 *
     /\
    2 3
```

- **Miért fontos?** Az AST a program logikai struktúráját képviseli, tiszteletben tartva a műveleti sorrendet és a zárójelezést. A program futtatása már ezen a fán fog végigmenni.

2.3. Kiértékelés - Evaluation:

Az AST felépítése után két fő út áll:

1. **Fordítás (Compilation):** Egy **fordítóprogram (compiler)** az AST-t egy másik nyelvre (jellemzően gépi kódra vagy egy alacsonyabb szintű nyelvre, mint a Java bytecode) fordítja le. A futtatás később történik, ezen az új, lefordított fájlra.
2. **Értelmezés (Interpretation):** Egy **értelmező (interpreter)** fogja az AST-t, és azonnal, "lépésről-lépésre" végrehajtja azt. Nem hoz létre új fájlt, hanem futásidőben értelmezi a fát.

A szakdolgozatom egy Interpretert valósít meg, mivel ez a megközelítés egyszerűbb, gyorsabb fejlesztést tesz lehetővé, és tökéletes egy dinamikus scriptnyelvhez.

3. A "MiniNyelv" Tervezése és Funkciói

A "MiniNyelv" egy egyszerű, dinamikusan típusos, sor-alapú scriptnyelv. Az alábbiakban a főbb funkcióit részletezzük.

3.1. Típusrendszer:

A nyelv két beépített típust kezel:

- **Szám (Number):** A nyelv minden számot double (lebegőpontos) típusként kezel.
- **Szöveg (String):** Idézőjelek (") között megadott karaktersorozatok.

3.2. Változók:

A változókat a var kulcsszóval lehet deklarálni és inicializálni. A már meglévő változók a var kulcsszó nélkül is kaphatnak új értéket.

```
# Változó deklaráció  
var x = 10  
var nev = "Teszt Elek"
```

```
# Értékadás (felülírás)  
x = (x + 5) * 2
```

3.3. Parancsok és Kifejezések:

- **print [kifejezés]:** Kiírja a konzolra a megadott kifejezés értékét.

```
print "Helló " + nev # Kiírja: Helló Teszt Elek  
print x / 2          # Kiírja: 15.0
```

- **Matematikai kifejezések:** Támogatja az alapvető +, -, *, / műveleteket, tiszteletben tartva a műveleti sorrendet és a zárójelezést. A + operátor stringek összefűzésére is képes.
- **Logikai kifejezések:** Támogatja az összehasonlító operátorokat: ==, !=, >, <, >=, <=.

3.4. Vezérlési Szerkezetek (IF):

A nyelv támogatja az if-then-else-endif szerkezeteket. A feltételnek egy logikai kifejezésnek kell lennie (amely 1.0-ra (igaz) vagy 0.0-ra (hamis) értékelődik ki).

```
var kor = 25

if (kor >= 18) then
    print "Felnőtt"
else
    print "Kiskorú"
endif
```

3.5. Kommentek:

A # jellel kezdődő sorok vagy sorrészek kommentnek minősülnek, az interpreter figyelmen kívül hagyja őket.

4. Implementáció Java nyelven:

A "MiniNyelv" interpreterének magja az **Interpreter Tervezési Minta** (Interpreter Design Pattern). Ennek lényege, hogy a nyelv nyelvtanának minden elemét (pl. "szám", "bináris művelet", "print parancs") egy külön Java osztály képviseli, amelyek mind egy közös interfészt valósítanak meg.

Az AST, amiről korábban írtam, valójában ezen Java objektumok összességéből épül fel.

4.1. Az Architektúra Fő Komponensei:

A projekt több, logikailag elkülönített komponensre lett bontva.

1. A Kifejezés Interfész – Expression osztályok: Ez a legfontosabb absztrakció. Minden, ami visszaad egy értéket (egy szám, egy string, egy változó értéke, egy matematikai művelet eredménye), az Expression.

- Expression.java

Java

```
public interface Expression {
    // Mivel van szám és string is, Object-et adunk vissza
    Object evaluate(Environment env);
}
```

- **Implementációk:**

- **NumberExpression:** Egy sima számot tárol (pl. 5). `evaluate()` önmagát adja vissza.
- **StringExpression:** Egy stringet tárol (pl. "hello").
- **VariableExpression:** Egy változó nevét tárolja (pl. x). `evaluate()` kikeresi a változó értékét a `Environment`-ből.
- **BinaryExpression:** Egy bal és egy jobb oldali `Expression`-t, valamint egy operátort tárol (pl. $5 + x$).
- **ComparisonExpression:** Hasonló a binárishoz, de összehasonlításokat végez.

2. A Parancs Interfész – Statement osztály: Minden, ami *végrehajt egy műveletet*, de nem ad vissza értéket (mint a `print` vagy az értékadás), az `Statement`.

- `Statement.java`

Java

```
public interface Statement {
    void execute(Environment env);
}
```

- **Implementációk:**

- **PrintStatement:** Tárol egy `Expression`-t. `execute()` kiértékeli és kiírja az eredményt.
- **VarAssignStatement:** Tárol egy változónevet és egy `Expression`-t. `execute()` kiértékeli a kifejezést és elmenti az eredményt a `Environment`-be.
- **IfStatement:** A legösszetettebb. Tárol egy `Expression`-t (a feltétel), egy `List<Statement>`-et (a "then" ág) és egy másik `List<Statement>`-et (az "else" ág).

3. Az Elemző – Parser osztály: A Parser.java osztály a rendszer "agya". **Tervezési döntés** volt, hogy a lexikális elemzést (lexing) és a szintaktikai elemzést (parsing) nem választottam szét mereven két külön komponensre, hanem a Parser felel mindkettőért.

- A parseStatementBlock() metódus rekurzívan olvassa a kódsorokat, és felismeri a blokk-szerkezeteket (if...endif).
- A parseExpression() metódus felel a kifejezések (pl. $(x + 5) * \text{"hello"}$) tokenizálásáért és AST-vé alakításáért. Ehhez egy **Recursive Descent** (rekurzív leszállásos) elemzési stratégiát használ, amely természetes módon kezeli a műveleti sorrendet és a zárójelezést.

4. A Környezet (Memória) – Environment osztály: Az Environment.java egy egyszerű osztály, ami egy HashMap<String, Object>-et burkol. Ez felel a változók tárolásáért futás közben. Az evaluate() és execute() metódusok ezt kapják paraméterként, hogy hozzáférjenek a változók állapotához.

5. A Főprogram - MiniInterpreter: Ez az alkalmazás belépési pontja. Felelős a .mini fájl nevének parancssori argumentumként való fogadásáért, a fájl beolvasásáért, a Parser példányosításáért, és a kapott List<Statement> (az AST gyökereinek listája) végrehajtásának elindításáért.

4.2. Tervezési Döntések:

- **Miért Object a típus?** A nyelv eleinte csak double típusokat kezelt. A String-ek bevezetésekor dönteni kellett: vagy bonyolult típusellenőrző rendszert építék, vagy egy egyszerűbb, "dinamikusabb" utat választok. Az Object használata (ahol az érték lehet Double vagy String) lehetővé tette, hogy a Environment és a kifejezések rugalmasan kezeljék mindkét típust. **Kompromisszum:** Emiatt a típusellenőrzést futásidőben (instanceof) kell elvégezni a műveleteket végző osztályokban (pl. BinaryExpression).
- **Miért if-then-else-endif?** Sok modern nyelv (pl. Java, C) kapcsos zárójeleket ({ }) használ a blokkok jelölésére. Az én parserem egyszerűsége érdekében egy egyértelműbb, kulcsszó-alapú lezárást (endif) választottam. Ez drasztikusan leegyszerűsítette a Parser logikáját, mivel nem kell a behúzásokat vagy a zárójelek párosítását figyelnie.

- **Miért egybeépített Lexer és Parser?** Egy professzionális fordítóprogramban a Lexer és a Parser szigorúan különálló komponensek. Egy ilyen kis nyelv esetében azonban ez felesleges bonyolítást jelentett volna. A kifejezések elemzését végző tokenizeExpression metódus (ami lényegében egy mini-lexer) beágyazása a Parser osztályba pragmatikus döntés volt.

4.3. A Projekt Osztályszerkezete és Fájljai:

A "MiniNyelv" interpreter 13 darab Java forrásfájlból épül fel. Az alábbiakban ezen osztályok és interfészek rövid ismertetése olvasható, funkcionális csoportokba rendezve.

A. Rendszerkomponensek (Core System):

Ezek az osztályok alkotják az interpreter "keretrendszerét", ők végzik a vezérlést és az adattárolást.

1. MiniInterpreter.java (Főprogram):

- Ez az alkalmazás belépési pontja (main metódus).
- Feladata a parancssori argumentumok ellenőrzése, a megadott .mini forrásfájl beolvasása, a Parser példányosítása, és a kapott parancslista végrehajtásának elindítása.
- Kezeli a fájlrendszerbeli (IO) kivételeket.

2. Parser.java (Elemző):

- A rendszer legösszetettebb osztálya. Felelős a nyers szöveges forráskód strukturált objektumokká (AST) alakításáért.
- Tartalmazza a lexikális elemzés logikáját (tokenizálás, stringek és operátorok felismerése) és a szintaktikai elemzést (műveleti sorrend, blokk-szerkezetek kezelése).
- Rekurzív leszállásos (Recursive Descent) algoritmust alkalmaz, amely egy felülről lefelé haladó elemző, amely a bemenetet rekurzív függvények halmaza alapján

dolgozza fel, ahol minden függvény egy nyelvtani szabálynak felel meg.
(*<https://www.geeksforgeeks.org/compiler-design/recursive-descent-parser/>)

3. Environment.java (Környezet):

- A program futásidejű állapotát tárolja.
- Belsőleg egy HashMap<String, Object>-et használ a változók neveinek és aktuális értékeinek (Szám vagy String) párosítására.
- Biztosítja a változók írását (setVariable) és olvasását (getVariable), valamint hibát dob ismeretlen változó esetén.

B. Interfészek (Interfaces):

Ezek definiálják az Interpreter Tervezési Minta közös szerződéseit.

4. Statement.java (Utasítás):

- Absztrakt interfész minden olyan nyelvi elemhez, amely cselekvést hajt végre, de nem rendelkezik visszatérési értékkel (pl. kiírás, értékadás, elágazás).
- Egyetlen metódusa: void execute (Environment env).

5. Expression.java (Kifejezés):

- Absztrakt interfész minden olyan nyelvi elemhez, amely kiértékelhető egy értékke (legyen az szám vagy szöveg).
- Egyetlen metódusa: Object evaluate(Environment env).

C. Utasítás Implementációk (Statements):

A Statement interfész konkrét megvalósításai.

6. VarAssignStatement.java:

- Felelős a változók deklarációjáért (var x = ...) és értékadásáért (x = ...).

- Kiértékeli a jobb oldali kifejezést, és az eredményt elmenti az Environment-be a megadott változónévvel.

7. **PrintStatement.java:**

- A print parancs megvalósítása.
- Kiértékeli a hozzá tartozó kifejezést, és az eredményt (legyen az szám vagy szöveg) kiírja a standard kimenetre (System.out).

8. **IfStatement.java:**

- A vezérlési szerkezet megvalósítása (if-then-else).
- Tárolja a feltételt (Expression) és két utasításlistát (igaz ág, hamis ág).
- Futásidőben ellenőrzi, hogy a feltétel érvényes szám-e, és ez alapján dönti el, melyik ágot hajtsa végre.

D. Kifejezés Implementációk (Expressions):

Az Expression interfész konkrét megvalósításai. Ezek alkotják a "leveleket" és a számítási csomópontokat az AST-ben.

9. **NumberExpression.java:**

- Primitív numerikus literált (pl. 10, 3.14) csomagol be egy objektumba.
- Visszatérési értéke önmaga (Double).

10. **StringExpression.java:**

- Szöveges literált (pl. "Hello World") csomagol be.
- Visszatérési értéke a tárolt String.

11. **VariableExpression.java:**

- Nem tárol értéket, csak egy változó nevét (azonosítót).
- Kiértékeléskor megkeresi és visszaadja a változó aktuális értékét az Environment-ből.

12. **BinaryExpression.java:**

- Kétooperandusú műveleteket valósít meg (+, -, *, /).
- Ez az osztály végzi a futásidejű típusellenőrzést is (pl. engedélyezi a stringek összefűzését +-szal, de tiltja a stringek szorzását).

13. **ComparisonExpression.java:**

- Logikai összehasonlításokat valósít meg (==, !=, <, >, stb.).
- Az eredményt a nyelv konvenciójának megfelelően 1.0 (igaz) vagy 0.0 (hamis) értékű Double-ként adja vissza. Kezeli a különböző típusok (szám vs. string) összehasonlításának szabályait is.

4.4. **Memóriakezelés és Állapottér (Environment):**

A "MiniNyelv" interpreteres működéséből adódóan a program futásidejű adatait (runtime data) a gazdanyelv, jelen esetben a Java memóriaterületén (Heap) tárolja.

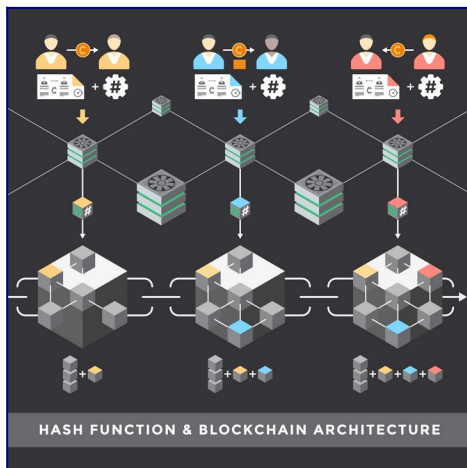
A nyelv memóriamodelljét az Environment osztály valósítja meg, amely a fordítóprogramok elméletében ismert **Szimbólumtábla (Symbol Table)** szerepét tölti be futásidőben.

4.4.1. **A Változók Tárolása**

A memóriaterület szerkezete egy kulcs-érték párokat tároló adatszerkezetre épül. A Java java.util.HashMap osztályát használtam erre a célra, amely $O(1)$ átlagos időbonyolultságú hozzáférést biztosít az adatokhoz. (* Az $O(1)$ jelölés a Nagy O (Big O) jelölésrendszerből származik, és konstans idejű (nagyon gyors, nem lassul be) műveletet jelöl. Ez a gyakorlatban azt jelenti, hogy a változók elérése, olvasása vagy írása a HashMap adatszerkezetben gyakorlatilag

ugyanannyi időt vesz igénybe, függetlenül attól, hogy a programban éppen 5 darab vagy 10 000 darab változó van definiálva.)

- **Kulcs (Key):** A változó azonosítója (String).
- **Érték (Value):** A változóban tárolt adat (Object). Mivel a nyelvünk dinamikusan típusos, ez az objektum lehet java.lang.Double (szám) vagy java.lang.String (szöveg). !!!Kép forrása:



Shutterstock

Példa a memória állapotára:

Tegyük fel, hogy a következő MiniNyelv kód fut:

```
var pi = 3.14  
var kozszonto = "Szia!"  
var x = 10
```

Ekkor az Environment objektum belső HashMap-je a következőképpen néz ki a Java memóriájában:

Változónév (Key)	Tárolt Érték (Value - Object)	Tényleges Típus
"pi"	3.14	Double
"koszonto"	"Szia!"	String
"x"	10.0	Double

4.4.2. Élettartam (Scope):

Jelen implementációban a memória **globális**. Az Environment példányosítása a program indulásakor (MiniInterpreter.main) történik, és a program futásának végéig létezik.

1. **Létrejött:** Amikor az interpreter elindul (new Environment()).
2. **Módosulása:** Amikor egy VarAssignStatement (var x = ... vagy x = ...) lefut.
3. **Megszűnése:** Amikor a program lefut, a Java Garbage Collector (Szemétgyűjtő) felszabadítja az Environment objektumot és a benne tárolt összes változót.

5. A Rendszer Fordítása és Futtatása

Ez a fejezet bemutatja, hogyan lehet a Java nyelven megírt interpretert lefordítani, illetve hogyan lehet vele egy saját, .mini kiterjesztésű forrásfájlt végrehajtani. A műveletek elvégzéséhez parancssori környezet (Terminal vagy Command Prompt) és telepített Java Development Kit (JDK) szükséges.

5.1. Könyvtárszerkezet:

A fordítás és futtatás előtt elengedhetetlen a megfelelő könyvtárszerkezet ellenőrzése. Mivel a projekt az org.example csomagot használja, a parancsokat mindig a **projekt gyökérkönyvtárából** (root directory) kell kiadni, amely tartalmazza az org mappát és a futtatandó .mini fájlt.

A helyes könyvtárszerkezet a következő:

ProjektGyökér/

├── teszt.mini (A futtatandó mini-program)

└── org/

 └── example/

 ├── MiniInterpreter.java

 ├── Parser.java

 └── Environment.java

└─ ... (többi .java fájl)

5.2. Az Interpreter Fordítása (Compilation):

Mielőtt használni tudnánk a "MiniNyelv" interpretert, a Java forráskódokat (.java) le kell fordítanunk a Java Virtuális Gép (JVM) által értelmezhető bájtkódra (.class).

A fordítás parancsa:

Bash

```
javac org/example/*.java
```

A parancs magyarázata:

- javac: A Java fordítóprogram meghívása.
- org/example/*.java: Utasítja a fordítót, hogy az org/example mappában található **összes** .java kiterjesztésű fájlt fordítsa le.

Ha a fordítás sikeres, a terminál nem ad vissza üzenetet, de az org/example mappában megjelennek a .class kiterjesztésű fájlok.

5.3. A Mini Program Futtatása (Execution):

A sikeres fordítás után az interpreter készen áll a használatra. A futtatáshoz a Java futtatókörnyezetet (java) használjuk, amelynek argumentumként megadjuk az interpreter főosztályát és a végrehajtandó .mini fájl nevét.

A futtatás parancsa:

Bash

```
java org.example.MiniInterpreter teszt.mini
```

A parancs részletes magyarázata:

1. java: Elindítja a Java Virtuális Gépet (JVM).

2. org.example.MiniInterpreter: Megnevezi a futtatandó főosztályt (Main Class). Fontos a **teljes minősített név** használata (csomagnév + osztálynév), ponttal elválasztva.
3. teszt.mini: **Parancssori argumentumként** adjuk át a feldolgozandó forrásfájl nevét. A MiniInterpreter osztály main metódusa ezt a fájlnévet fogja beolvasni és átadni a Parsernek.

5.4. Várt Kimenet

Ha a teszt.mini fájl helyes szintaxisú kódot tartalmaz, az eredmény azonnal megjelenik a konzolon.

Példa a sikeres futtatásra:

teszt.mini fájl:

--- MiniNyelv futtatása: teszt.mini ---

```
var a = 20  
var b = 10
```

```
print a * b # Eredmény: 200
```

```
if (a > b) then  
  print "A nagyobb mint B"
```

```
  if ( (a / b) == 2 ) then  
    print "A kétszerese B-nek"  
  endif  
else  
  print "B nagyobb mint A"  
endif
```

```
print "Program vége!"
```

--- Futtatás vége ---

Hiba esetén (pl. szintaktikai hiba a .mini fájlban vagy hiányzó fájl) a program kivételt (Exception) dob, és hibaüzenettel tájékoztatja a felhasználót a probléma jellegéről.

6. Összegzés és további fejlesztési lehetőségek

A "MiniNyelv" szakdolgozatom sikeresen demonstrálta egy programozási nyelv interpreterének alapvető felépítését és működését. A jelenlegi implementáció egy stabil alapot biztosít, amely számos irányba bővíthető:

1. **Ciklusok:** while vagy for ciklusok implementálása, hasonlóan az IfStatement-hez, blokk-kezeléssel.
2. **Funkciók:** Saját funkciók definiálása (function nev(arg1, arg2) ... endfunc) és meghívása. Ez egy jelentős lépés, amihez "hívási verem" (Call Stack) és saját "scope" (hatókör) kezelés szükséges.
3. **Logikai Operátorok:** and és or operátorok bevezetése a feltételekbe.
4. **Több Típus:** boolean (igaz/hamis) vagy null típusok explicit bevezetése.
5. **Tömbök/Listák:** Összetett adatstruktúrák kezelése.

A szakdolgozatommal szerettem volna bemutatni, hogy a szoftverfejlesztés absztrakcióinak megértéséhez néha a legjobb út az, ha mi magunk építjük fel azokat a legalapoktól, mert így sokkal érthetőbb lesz majd a későbbiekben a választott programozási nyelv működése és szintaktikája is illetve maga a programozás is.